BECS 22623
Implementation of Numerical Methods using Appropriate software

Department of Mathematics
Faculty of Science
University of Kelaniya

Outline

1. Fixed-Point Iteration
2. Newton-Raphson

Simple Fixed-Point Iteration

Fixed-point iteration is a numerical method used to solve equations of the form $x = g(x)$. It is an iterative process where an initial guess x_0 is refined by repeatedly applying a function $g(x)$ to approach a solution.

- **Key Idea**
  - A fixed point of a function $g(x)$ is a value x^* such that $g(x^*) = x^*$
  - To find this point, the iteration formula is used:

$$x_{n+1} = g(x_n)$$

where x_n is the current approximation and x_{n+1} is the next approximation.

Steps in Fixed-Point Iteration

1. **Rearrange the equation:** Start with an equation $f(x) = 0$ and rewrite it in the form $x = g(x)$, where $g(x)$ is a continuous function.
2. **Choose an initial guess:** Select an initial approximation x_0
3. **Iterate:** Apply the iteration formula $x_{n+1} = g(x_n)$ repeatedly.
4. **Convergence check:** Stop the iteration when $|x_{n+1} - x_n| < \epsilon$, where ϵ is a small tolerance, or after a maximum number of iterations.

Convergence of Fixed-Point Iteration

- Fixed-point iteration converges if the function $g(x)$ satisfies the contraction condition:

$$|g'(x)| < 1 \text{ for all x in the interval of interest.}$$

- If $|g'(x)| \geq 1$, the method may diverge.

Example

Example

Use simple fixed-point iteration to locate the root of

$$f(x) = e^{-x} - x.$$

The function can be separated directly and expressed in the form

$$x = g(x) = e^{-x}$$

so that $x_{i+1} = g(x_i) = e^{-x_i}$. Starting with an initial guess of $x_0 = 0$, this iterative equation can be applied to compute:

Example-Fixed-Point Iteration Code

```
% Fixed Point Iteration in MATLAB

% Define the function g(x) and the initial guess
g = @(x) exp(-x); % Example: g(x) = e^{-x}
x0 = 0;           % Initial guess
tol = 1e-6;       % Tolerance for convergence
maxIter = 100;    % Maximum number of iterations

% Initialize variables
x = x0;
iter = 0;

fprintf('Iter\t x\t\t g(x)\t\t Error\n');
fprintf('-----------------------------------------\n');

while iter < maxIter
    iter = iter + 1;
    x_new = g(x);                  % Perform the iteration: x_(n+1) = g(x_n)
    error = abs(x_new - x);        % Compute the error
    fprintf('%d\t %.6f\t %.6f\t %.6f\n', iter, x, x_new, error);

    if error < tol                 % Check for convergence
        fprintf('Converged to fixed point %.6f after %d iterations.\n', x_new, iter);
        break;
    end
    x = x_new;                     % Update x for the next iteration
end

if iter == maxIter
    fprintf('Failed to converge after %d iterations.\n', maxIter);
end
```

```
Iter      x                g(x)             Error
-----------------------------------------
1         0.000000         1.000000         1.000000
2         1.000000         0.367879         0.632121
3         0.367879         0.692201         0.324321
4         0.692201         0.500474         0.191727
5         0.500474         0.606244         0.105770
6         0.606244         0.545396         0.060848
7         0.545396         0.579612         0.034217
8         0.579612         0.560115         0.019497
9         0.560115         0.571143         0.011028
10        0.571143         0.564879         0.006264
11        0.564879         0.568429         0.003549
12        0.568429         0.566415         0.002014
13        0.566415         0.567557         0.001142
14        0.567557         0.566909         0.000648
15        0.566909         0.567276         0.000367
16        0.567276         0.567068         0.000208
17        0.567068         0.567186         0.000118
18        0.567186         0.567119         0.000067
19        0.567119         0.567157         0.000038
20        0.567157         0.567135         0.000022
21        0.567135         0.567148         0.000012
22        0.567148         0.567141         0.000007
23        0.567141         0.567145         0.000004
24        0.567145         0.567142         0.000002
25        0.567142         0.567144         0.000001
26        0.567144         0.567143         0.000001
Converged to fixed point 0.567143 after 26 iterations.
```

Exercise

Problem

Employ fixed-point iteration to locate the root of

$$f(x) = \sin(\sqrt{x}) - x.$$

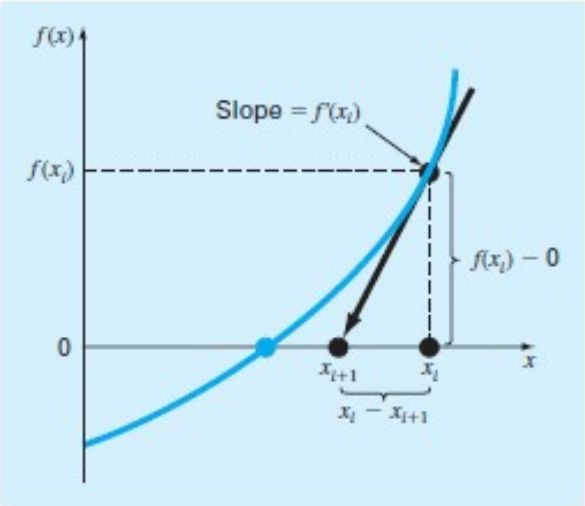
Use an initial guess of $x_0 = 0.5$ and iterate until $\epsilon_a \leq 0.01\%$.

Newton-Raphson

The Newton-Raphson method is an efficient and widely-used iterative numerical technique for finding approximate solutions (roots) of an equation $f(x) = 0$.

Newton-Raphson formula

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$



Algorithm for Newton-Raphson

INPUTS: f(x): nonlinear function as a function handle

fp(x): the derivative of the nonlinear function as a function handle

seed: Initial value to start the method

maxitr: maximum number of interations

m = multiplicity of the root

REPEAT: seed1 = seed - m*f(seed)/fp(seed)

seed = seed1;

UNTIL: maximum number of interations is reached

OUPUT: Approximate root

Algorithm for Newton-Raphson

Check to see if the initial seed is a good a approximation to the root. If it is, stop further execution with 'return' otherwise continue with execution. The following lines of code do this

if abs(f(seed)) $< 0.5 \times 10^{-16}$

disp(' Your seed is a good approximation to the root')

return

end

MATLAB code for Newton-Raphson

```
% newton's method

function [root,root_vector] = Newton(f,fp,seed,maxitr,m)

if abs( f(seed) ) <  0.5*10^(-16)
    disp(' Your seed is a good approximation to the root')
    return
end

root_vector = zeros(1,maxitr);

for i =1:maxitr
    seed1 = seed - m*f(seed)/fp(seed);
    seed = seed1;

    root_vector(i) =seed1;
end

root = seed1;


end
```

Example

Use your Newton function developed in previous slide to approximate the root, $\sqrt{2}$ of $f(x) = x^2 - 2$, near $x = 1$ using 10 iterations. What is the absolute error after 10 iterations? How many correct decimal places does your approximate root have?

Solution

```
f = @(x) x^2-2
fp= @(x) 2*x
seed=1

maxitr = 10
m=1
root = Newton(f,fp,seed,maxitr,m)

absolute_error = sqrt(2)-root
```

Exercises

Problem

Use the Newton-Raphson method to estimate the root of $f(x) = e^{-x} - x$ employing an initial guess of $x_0 = 0$.

Problem

Use

- fixed-point iteration and
- the Newton-Raphson method

to determine a root of $f(x) = -0.9x^2 + 1.7x + 2.5$ using $x_0 = 5$. Perform the computation until ϵ_a is less than $\epsilon_s = 0.01\%$.

Optimization

Root finding involves searching for the location where the function equals zero. In contrast, optimization involves searching for the function's extreme points.

The optimums are x value where the derivative $f'(x)$ is equal to zero. Additionally, the second derivative, $f''(x)$, indicates whether the optimum is a minimum or a maximum:

If $f''(x) < 0$, the point is a maximum; If $f''(x) > 0$, the point is a minimum.

Optimization

Now, understanding the relationship between roots and optima would suggest a possible strategy for finding the latter.

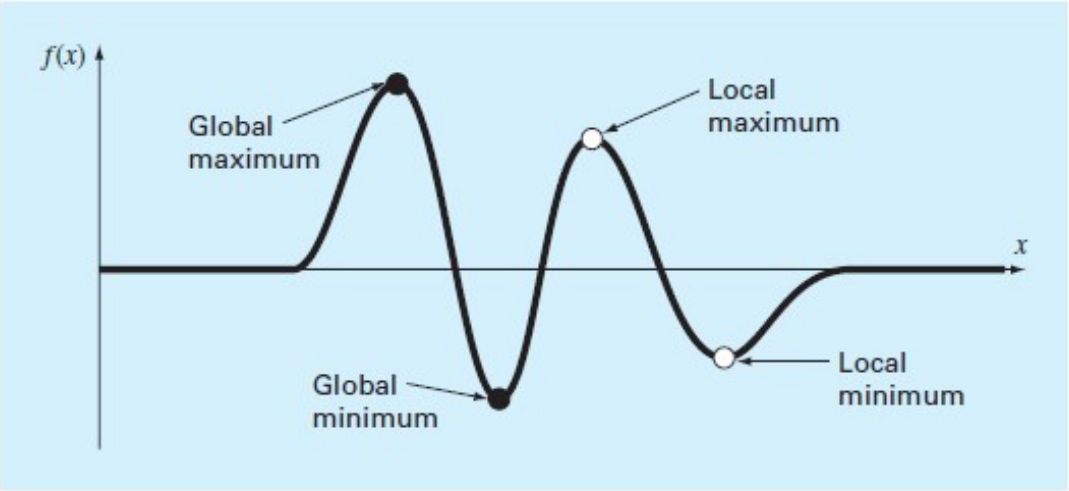
That is, you can differentiate the function and locate the root (i.e., the zero) of the new function.

In fact, some optimization methods do just this by solving the root problem: $f'(x) = 0$.

One-Dimensional Optimization

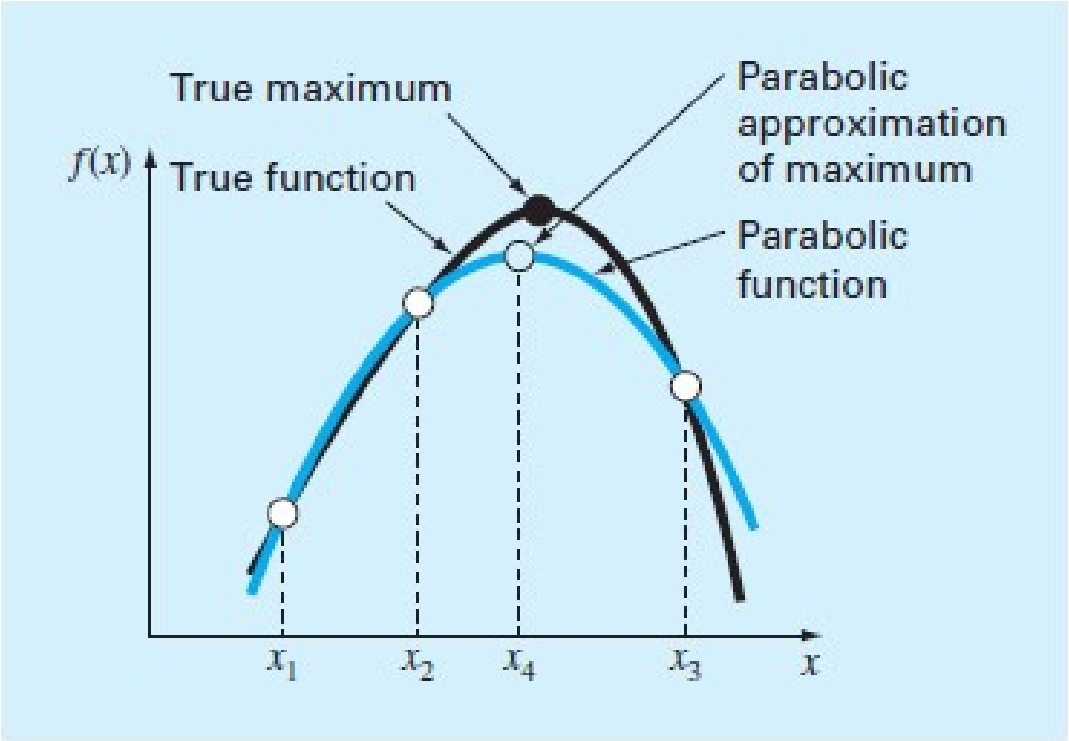
This section will describe techniques to find the minimum or maximum of a function of a single variable $f(x)$.

A global optimum represents the very best solution. A local optimum, though not the very best, is better than its immediate neighbors. Cases that include local optima are called multimodal. In such cases, we will almost always be interested in finding the global optimum.



Parabolic Interpolation

Parabolic interpolation takes advantage of the fact that a second order polynomial often provides a good approximation to the shape of $f(x)$ near an optimum:



Parabolic Interpolation

Just as there is only one straight line ($f(x) = ax + b$) connecting two points, there is only one parabola ($f(x) = ax^2 + bx + c$) connecting three points. Thus, if we have three points that jointly bracket an optimum, we can fit a parabola to the points.

Then we can differentiate it, set the result equal to zero, and solve for an estimate of the optimal x. It can be shown through some algebraic manipulations that the result is

$$x_4 = x_2 - \frac{1}{2}\frac{(x_2 - x_1)^2[f(x_2) - f(x_3)] - (x_2 - x_3)^2[f(x_2) - f(x_1)]}{(x_2 - x_1)[f(x_2) - f(x_3)] - (x_2 - x_3)[f(x_2) - f(x_1)]}$$

where x_1, x_2, and x_3 are the initial guesses, and x_4 is the value of x that corresponds to the optimum value of the parabolic fit to the guesses.

Parabolic Interpolation

Problem

Use parabolic interpolation to approximate the minimum of

$$f(x) = \frac{x^2}{10} - 2\sin x$$

with initial guesses of $x_1 = 0$, $x_2 = 1$, and $x_3 = 4$.

Parabolic Interpolation

Solution:

The function values at the three guesses can be evaluated:

$$x_1 = 0 \qquad f(x_1) = 0$$
$$x_2 = 1 \qquad f(x_2) = -1.5829$$
$$x_3 = 4 \qquad f(x_3) = 3.1136$$

So that

$$x_4 = 1 - \frac{1}{2}\frac{(1-0)^2[-1.5829-3.1136]-(1-4)^2[-1.5829-0]}{(1-0)[-1.5829-3.1136]-(1-4)[-1.5829-0]} = 1.5055$$

which has a function value of $f(1.5055) = -1.7691$.

Parabolic Interpolation

Next, we determine which point should be discarded.

Because the function value for the new point is lower than for the intermediate point (x_2) and the new x value is to the right of the intermediate point, the lower guess (x_1) is discarded.

Therefore, for the next iteration:

$$\begin{aligned} x_1 &= 1 & f(x_1) &= -1.5829 \\ x_2 &= 1.5055 & f(x_2) &= -1.7691 \\ x_3 &= 4 & f(x_3) &= 3.1136 \end{aligned}$$

Substituting into the corresponding equation yields $x_4 = 1.4903$, which has a function value of $f(1.4903) = -1.7714$.

Parabolic Interpolation

The process can be repeated, with the results tabulated here:

i	x_1	$f(x_1)$	x_2	$f(x_2)$	x_3	$f(x_3)$	x_4	$f(x_4)$
1	0.0000	0.0000	1.0000	−1.5829	4.0000	3.1136	1.5055	−1.7691
2	1.0000	−1.5829	1.5055	−1.7691	4.0000	3.1136	1.4903	−1.7714
3	1.0000	−1.5829	1.4903	−1.7714	1.5055	−1.7691	1.4256	−1.7757
4	1.0000	−1.5829	1.4256	−1.7757	1.4903	−1.7714	1.4266	−1.7757
5	1.4256	−1.7757	1.4266	−1.7757	1.4903	−1.7714	1.4275	−1.7757

Thus, within five iterations, the result is converging rapidly on the true value of -1.7757 at $x = 1.4276$.

MATLAB Function: fminbnd

The built-in Matlab function **fminbnd** combines the slow, but dependable goldensection search with the faster converging parabolic interpolation method to solve for the minimum of a function.

A simple expression of its syntax is:

[xmin, fval] = fminbnd(func,x1,x2)

where **xmin** and **fval** are the location and value of the minimum, **func** is the name of the function being evaluated, and **x1** and **x2** are the bounds of the interval being searched.

MATLAB Function: fminbnd

The following is a Matlab session to solve for the minimum of the function, considered in the previous example, employing **fminbnd**:

```
>> fun=@(x) x^2/10-2*sin(x);
>> [x, fval] = fminbnd(fun,0,4)

x =

    1.4275


fval =

   -1.7757

fx >>
```

MATLAB Function: fminbnd

How to use **fminbnd** to find the maximum?

```
>> fun = @(x) x^2/10-2*sin(x);
>> negfun = @(x) -fun;
>> [xmax,negfval]=fminbnd(fun,0,4)

xmax =

    1.4275


negfval =

   -1.7757

>> ymax = - negfval

ymax =

    1.7757
```

Department of Mathematics

Faculty of Science, University of Kelaniya.